Department of Applied Computing and Artificial Intelligence

Faculty of Computing

UNIVERSITI TEKNOLOGI MALAYSIA

PROGRAMMING TECHNIQUE II
(SECJ1023**)**

SEMESTER II 2023/2024

Project and Assignment Software Documentation
Equinox Banking System

By
Chew Chiu Xian (A23CS0061)
Lau Yan Kai (A23CS0098)
Lee Yin Shen (A23CS0236)

SECTION 02

Lecturer:
Ts. Dr. Johanna binti Ahmad

Date
20 June 2024

PART 1: INTRODUCTION

1.1 Synopsis Project

In this project, our team will develop a banking system called Equinox Bank System. The system offers a variety of essential banking functions for both users and administrators.

For users, these include depositing and withdrawing money, transferring funds to other users, paying bills, checking account balances, and adjusting account settings.

Administrators have the ability to view the complete list of users, access detailed user information, and review user transaction histories.

Access to the system requires both users and administrators to enter their account username and security PIN for verification. Once verified, they can access a main menu where they can select their desired operation by entering the corresponding number.

Overall, the Equinox Bank System aims to deliver an efficient and user-friendly banking platform for managing all banking activities.

1.2 Objective of The Project

- Customer able to transfer money, deposit money, withdraw money, and make payment for their bill.
- Customer able to check their account balance and change credit card settings
- Admin able to oversee user management tasks such as viewing user lists, accessing detailed user information, and reviewing transaction histories.
- Implement secure authentication using account usernames and security PINs for both users and administrators.

PART 2: SYSTEM ANALYSIS AND DESIGN (USE CASE, FLOWCHART AND CLASS DIAGRAM)

2.1 System Requirements

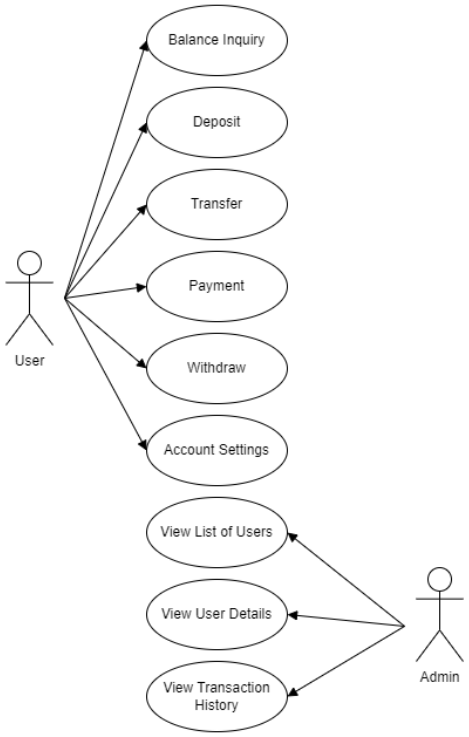

Figure 1 : Use Case Diagram for Equinox Banking System

Detail Description for Each Use Cases

The system has 9 main use cases.

Use Case	**Purpose**
Balance Inquiry	To check the current balance of user's account to stay updated on available funds.
Deposit	To add funds to user's account.
Transfer	To move funds between accounts, either within the same bank or to external accounts.
Payment	To settle bills or invoices directly from user's account to a service provider.
Withdrawal	To take out funds from user's account.
Account Settings	To customise and manage account preferences, such as security settings and contact details.
View List of Users	To display a list of all registered users for administrative or informational purposes.
View User Details	To access and review specific information about a particular user, such as account details and personal data.
View Transaction History	To review past transactions for monitoring spending, or auditing accounts.

2.2 System Design

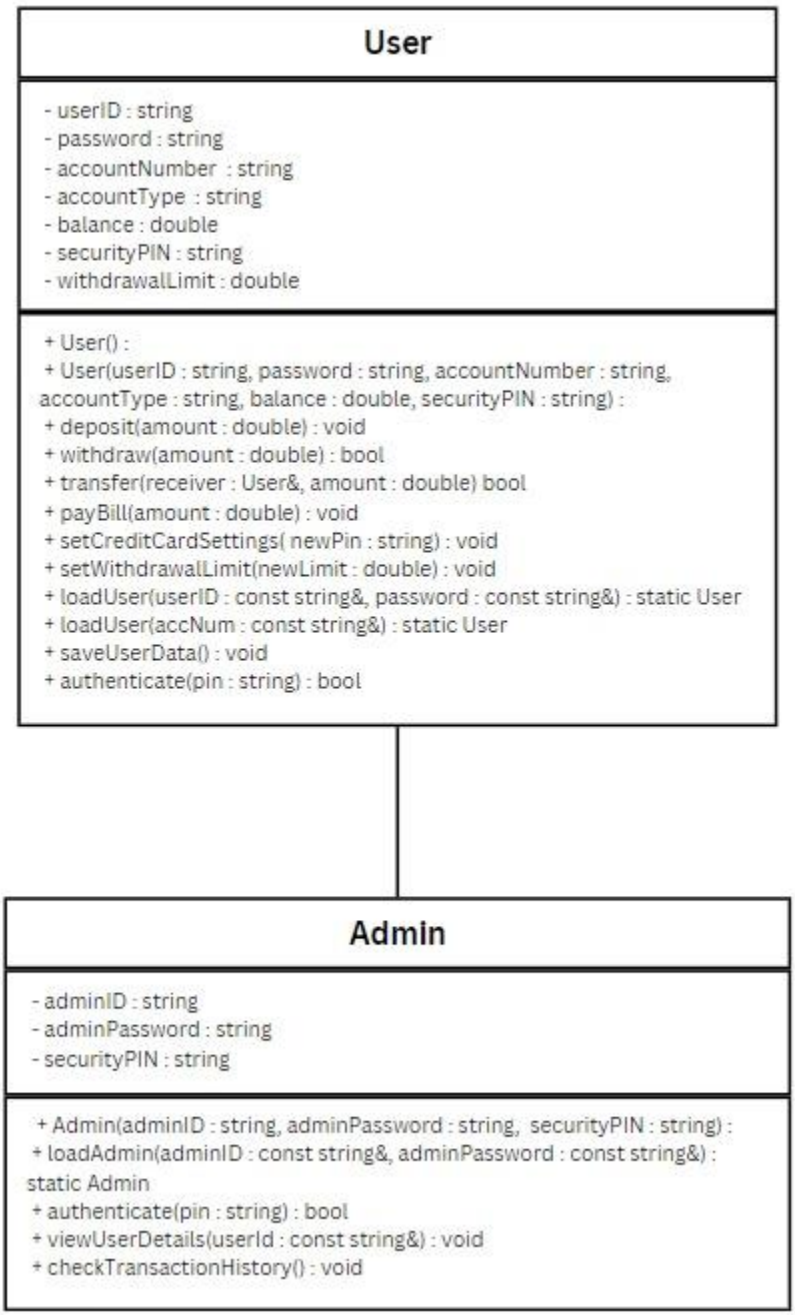

Figure 2 : Class Diagram for Equinox Banking System

PART 3: SYSTEM PROTOTYPE

EQUINOX BANKING SYSTEM

== Login ==

1. *Login (User)*
2. *Login (Admin)*
3. *Exit*

Enter your choice >> ***1***

Enter User ID >> ***johndoe2024***
Enter Password >> ***SysAdm!***
Enter Security PIN >> ***264891***

Login Successful! Redirecting to Main Menu…

Screen 1: *Login Interface*

Screen 1: The user must select 1 for User Login, 2 for Admin Login, or 3 to Exit. Input must be an integer from 1-3; invalid entries will prompt an error, and the screen reappears.

EQUINOX BANKING SYSTEM

== Main Menu ==

1. *Account Details*
2. *Deposit*
3. *Transfer*
4. *Payment*
5. *Withdrawal*
6. *Account Settings*
7. *Exit*

Enter your choice >> ***1***

Screen 2: *Main Menu for User Account*

Screen 2: The user is presented with a list of banking operations, such as viewing account details, making deposits, transferring funds, making payments, withdrawing money, and adjusting account settings. Users select an option by entering a number from 1 to 7. Invalid entries prompt an error, and the screen reappears for a correct input.

EQUINOX BANKING SYSTEM

== Account Details ==

USER ID >> johndoe2024
Account Number >> 890-1234-569
Account Type >> Savings
Balance >> $ 2500.00

Press any key to go back to main menu……

Screen 3: *User's Account Details*

Screen 3: The screen displays detailed information about the user's account, including User ID, Account Number, Account Type, and current Balance. This summary provides a quick overview of account details. Users can return to the main menu by pressing any key.

EQUINOX BANKING SYSTEM

== Deposit ==

Enter Account to Deposit into >> ***234-5678-907***
Enter Amount to Deposit >> $ ***200***
Please scan the ATM QR code with your Device
ATM Serial Number >> 14513
Confirm Connection (y/n) >> ***y***
Successful Connection.
Please Insert your Money.

Success!
Do you want to print your receipt? (y/n) >> ***y***

Screen 4: *Deposit*

Screen 4: This screen facilitates the deposit process by prompting the user to enter the account number and the amount to be deposited. Users are then instructed to scan the ATM QR code and confirm the connection with the ATM serial number. After confirming, they can insert their money. A success message is displayed, and users are given the option to print a receipt by entering 'y' or 'n'.

EQUINOX BANKING SYSTEM

Transaction Date :	*Jun 20 2024*
Transaction Time:	*15:45:41*
Account Deposited:	*234-5678-907*
Amount Deposited:	*$200.00*
Denominations:	
RM 100:	*2*
RM 50:	*0*
RM 20:	*0*
RM 10:	*0*
RM 5:	*0*
RM 1:	*0*

Screen 5: *Deposit Receipt*

Screen 5: The screen provides a detailed summary of the deposit transaction, including the date and time, the account number where the deposit was made, the deposited amount, and the denominations of the deposited fund. This receipt serves as a confirmation and record of the transaction.

EQUINOX BANKING SYSTEM

== Transfer ==

Enter Receiver's Account Number >> ***789-0123-464***
Enter amount to transfer (You may transfer up to $ 14030.00): >> $ ***200***
Enter your Security PIN >> ***140682***
Transfer Successful.

Do you want to print your receipt? (y/n) >> ***y***

Screen 6: *Transfer*

Screen 6: This screen guides the user through the process of transferring funds by prompting for the receiver's account number, transfer amount and security PIN. After entering the details, the transfer is confirmed as successful, and the user is given the option to print a receipt by entering 'y' or 'n'.

```
                    EQUINOX BANKING SYSTEM


                    Transaction Date :          Jun 20 2024
                    Transaction Time:           15:45:41
                    Sender Account Number:      234-5678-906
                    Receiver Account Number:    789-0123-464
                    Amount Transferred:         $ 200.00
                    New Balance:                $ 13840.00


 Press any key to go back to main menu...
```

Screen 7: *Transfer Receipt*

Screen 7: This screen provides a detailed summary of the fund transfer transaction, displaying the transaction date and time, sender's and receiver's account numbers, the amount transferred, and the new balance in the sender's account. Users can press any key to return to the main menu.

```
                    EQUINOX BANKING SYSTEM

                          == Payment ==

          Payment Options:
                    1. Electrical Bill
                    2. Water Bill
                    3. Telephone Bill
                    4. Credit Card Bill

                    Enter your choice >> 1
```

Screen 8: *Payment*

Screen 8: This screen provides users with payment options for various bills, including Electrical, Water, Telephone, and Credit Card bills. Users select the desired option by entering the corresponding number. Invalid entries will prompt an error, and the screen will reappear for a correct input.

```
EQUINOX BANKING SYSTEM

== Payment ==

Bill Number          >> 31232
Total Bill           >> RM 74.00
Utility Company      >> Tenaga Nasional Berhad (TNB)
Confirmation (y/n)   >> y
```

Screen 9: *Electrical Bill Payment*

Screen 9: This screen facilitates the payment of an electrical bill. Users are prompted to enter the bill number, view the total bill amount, and confirm the utility company, which in this case is Tenaga Nasional Berhad (TNB). The user confirms the payment by entering 'y' to proceed with the transaction.

```
EQUINOX BANKING SYSTEM


Transaction Date :      Jun 20 2024
Transaction Time:       15:45:41
Account Number:         234-5678-907
Bill Number:            31232
Amount Paid:            RM 74.00
Utility Company     >>  Tenaga Nasional Berhad (TNB)
Thank you for your payment.
Please keep this receipt for future reference.

New balance  >>  $13766.00

Press any key to go back to main menu...
```

Screen 10: *Bill Payment Receipt*

Screen 10: This screen provides a detailed receipt for the bill payment, showing the transaction date and time, account number, bill number, amount paid, and the utility company (Tenaga Nasional Berhad). It includes a thank-you note and advises the user to keep the receipt for future reference. The new account balance is also displayed. Users can press any key to return to the main menu.

EQUINOX BANKING SYSTEM

== Withdrawal ==

Enter Amount to Withdraw >> ***$ 200***
(You may withdraw up to $ 2000)

Please scan the ATM QR code with your device.
ATM Serial Number: 21426
Confirm Connection (y/n) >> ***y***

Connection Successful.
Enter your Security PIN >> ***140682***

Withdrawal successful!
New Balance >> $ 13764.00

Do you want to print your receipt? (y/n) >> ***y***

Screen 11: *Withdrawal*

Screen 11: This screen guides the user through the withdrawal process. Users enter the desired withdrawal amount, up to a specified limit, and scan the ATM QR code. After confirming the ATM connection and entering their security PIN, the withdrawal is processed. The screen displays the new account balance and offers the option to print a receipt by entering 'y' or 'n'.

EQUINOX BANKING SYSTEM

Transaction Date :	*Jun 20 2024*
Transaction Time:	*15:45:41*
Account Deposited:	*234-5678-907*
Amount Withdrawal:	*$200.00*
Denominations:	
RM 100:	*2*
RM 50:	*0*
RM 20:	*0*
RM 10:	*0*
RM 5:	*0*
RM 1:	*0*

Screen 12: *Withdrawal Receipt*

Screen 12: This screen provides a detailed receipt for the withdrawal transaction, including the transaction date and time, the account from which the withdrawal was made, the withdrawn amount, and the number of denominations received. This receipt serves as confirmation of the transaction details for the user's records.

EQUINOX BANKING SYSTEM

== Account Settings ==

Choose an Option:

1. *Change Credit Card PIN*
2. *Change Withdrawal Limit*

Enter your choice >> ***1***

Screen 13: *Account Settings*

Screen 13: This screen offers users two options to manage their account settings. They can choose to change their credit card PIN for enhanced security (Option 1) or adjust their withdrawal limit as needed (Option 2). Users select their preferred option by entering the corresponding number, facilitating convenient management of their banking preferences directly from this interface.

EQUINOX BANKING SYSTEM

== Account Settings ==

Enter new PIN: ***193852***

Credit Card PIN Updated!
New PIN >> 193852

Press any key to go back to main menu...

Screen 14: *New PIN Settings*

Screen 14: This screen confirms the successful update of the credit card PIN to the new PIN entered by the user. Users are prompted to press any key to return to the main menu.

EQUINOX BANKING SYSTEM

== Account Settings ==

Enter new Withdrawal Limit: ***$ 688***

ATM Withdrawal Limit Updated!
New Limit >> $ 688

Press any key to go back to main menu...

Screen 15: *New Withdrawal Limit Settings*

Screen 15: This screen confirms the successful update of the ATM withdrawal limit to the new limit specified by the user. Users are prompted to press any key to return to the main menu.

EQUINOX BANKING SYSTEM

== Main Menu ==

1. *View list of registered users*
2. *View user details*
3. *Check transaction history*
4. *Exit*

Enter your choice >> ***1***

Screen 16: *Main Menu for Admin Account*

Screen 16: This menu provides administrators with functions for managing the banking system. They can view a list of registered users (Option 1), access detailed information about individual users (Option 2), and review transaction histories (Option 3) for monitoring and auditing purposes. Option 4 allows administrators to exit the application. Each option is accessed by entering the corresponding number, enabling administrators to oversee and maintain banking operations from this centralised interface.

EQUINOX BANKING SYSTEM

== Users Accounts ==

Account Number	Account Type	Balance
123-4567-890	Savings	$38547.00
234-5678-903	Savings	$14379.00
456-7890-124	Savings	$6980.00
234-5678-905	Savings	$8500.00
890-1234-569	Savings	$2500.00
234-5678-907	Savings	$1750.00
012-3456-781	Savings	$ 800.00
234-5678-906	Savings	$ 50.00
789-0123-464	Savings	$1450.00

Press any key to go back to main menu...

Screen 17: *View Users Accounts*

Screen 17: This screen provides an overview of user accounts within the system. Admin can press any key to return to the main menu.

EQUINOX BANKING SYSTEM

== User Details ==

Enter User ID to view details: ***techsupport***

User ID >> techsupport
Account Number >> 789-0123-464
Account Type >> Savings
Balance >> $1450.00
Security PIN >> 276593

Press any key to go back to main menu...

Screen 18: *View User Details*

Screen 18: This screen allows administrators to retrieve and view detailed information about a specific user by entering their User ID. It displays the User ID, Account Number, Account Type, current Balance, and Security PIN associated with the user. Administrators can press any key to return to the main menu after viewing the details.

EQUINOX BANKING SYSTEM

== Transaction History ==

```
-------------------------------------------------------------------
Sender Account | Date       | Time     | Amount  | Recipient Account
-------------------------------------------------------------------
   123-4567-890 |  2024-6-19 | 18:18:35 | 2500.00 |      234-5678-903
-------------------------------------------------------------------
```

Press any key to go back to main menu...

Screen 19: *Transaction History*

Screen 19: This screen allows administrators to access and review the transaction history within the banking system. Administrators can press any key to return to the main menu after reviewing the transaction details.

PART 4 : APPENDIX

Source code for main function

```
#include <iostream>
#include <fstream>
#include <sstream>
#include <string>
#include <ctime>
#include <cstdlib>
#include <chrono>
#include <thread>
#include <conio.h>
#include "User.h"
#include "Admin.h"
using namespace std;

void showMenu() {
    cout << "Main Menu:\n";
    cout << "1. Account Details\n";
    cout << "2. Deposit\n";
    cout << "3. Transfer\n";
    cout << "4. Payment\n";
    cout << "5. Withdraw\n";
    cout << "6. Account Setting\n";
    cout << "7. Exit\n";
    cout << "Enter your choice: ";
}

void waitforUser() {
    cout << "\n\n\nPress any key to go back to main menu...";
    _getch();
    system("cls");
}

void accountDetails(User &user) {
    system("cls");
    cout << "----------------------Account Details---------------
----------\n";
    cout << "Equinox Bank Berhad\n\n";
    cout << "== Account Details ==\n\n\n";
    cout << "User ID: " << user.userID << "\n";
    cout << "Account Number: " << user.accountNumber << "\n";
    cout << "Account Type: " << user.accountType << "\n";
    cout << "Balance: $" << fixed << setprecision(2) <<
user.balance << "\n";
    waitforUser();
}

void deposit(User &user) {
    system("cls");
    cout << "----------------------Deposit----------------------
---\n";

    srand(time(0)); // Seed the random number generator with the
current time
```

```
53      int atmSerialNumber = rand() % 10000000000; // Generate a
54  random 10-digit number
55
56      double amount;
57      string accountNumber;
58      cout << "Enter account to deposit into: ";
59      cin >> accountNumber;
60      cout << "Enter amount to deposit: $";
61      cin >> amount;
62      cout << "Please scan the ATM QR code with your device.\n";
63      cout << "ATM Serial Number: " << atmSerialNumber << endl; //
64  Display the random ATM serial number
65      cout << "Confirm connection (y/n): ";
66      char confirm;
67      cin >> confirm;
68
69      if (confirm == 'y' || confirm == 'Y') {
70          cout << "Successful connection.\n";
71          cout << "Please insert your money.\n";
72          user.deposit(amount);
73          cout << "Success!\n";
74          cout << "Do you want to print the receipt? (y/n): ";
75          cin >> confirm;
76          if (confirm == 'y' || confirm == 'Y') {
77              cout << "\nTransaction Date: " << __DATE__ << "\n";
78              cout << "Transaction Time: " << __TIME__ << "\n";
79              cout << "Account Deposited: " << accountNumber <<
80  "\n";
81              cout << "Amount Deposited: $" << amount << "\n";
82
83              int intAmount = static_cast<int>(amount);
84              int num100 = intAmount / 100;
85              intAmount %= 100;
86              int num50 = intAmount / 50;
87              intAmount %= 50;
88              int num20 = intAmount / 20;
89              intAmount %= 20;
90              int num10 = intAmount / 10;
91              intAmount %= 10;
92              int num5 = intAmount / 5;
93              intAmount %= 5;
94              int num1 = intAmount;
95
96              cout << "Denominations: \n";
97              cout << "RM100: " << num100 << "\n";
98              cout << "RM50: " << num50 << "\n";
99              cout << "RM20: " << num20 << "\n";
100             cout << "RM10: " << num10 << "\n";
101             cout << "RM5: " << num5 << "\n";
102             cout << "RM1: " << num1 << "\n";
103
104             waitforUser();
105         }
106     } else {
107         cout << "Connection failed.\n";
108     }
```

```
}

void transfer(User &user) {
    string ReceiverAccNum, pin;
    double amount;

    system("cls");
    cout << "----------------------Transfer----------------------
----\n";

ReceiverAccNum:
    cout << "Enter receiver's account number: ";
    cin >> ReceiverAccNum;

    User receiver = User::loadUser(ReceiverAccNum);
    if (receiver.accountNumber.empty()) {
        cout << "Receiver not found.\n";
        goto ReceiverAccNum;
    }

TransferringAmount:
    cout << "Enter amount to transfer (You may transfer up to $ "
<< fixed << setprecision(2) << user.balance - 10 << "):   $";
    cin >> amount;

    if (user.balance - 10 < amount) {
        cout << "Insufficient balance for transfer.\n";
        goto TransferringAmount;
    }

EnteringPIN:
    cout << "Enter your security PIN: ";
    cin >> pin;

    if (!user.authenticate(pin)) {
        cout << "Security PIN authentication failed.\n";
        goto EnteringPIN;
    }

    if (user.transfer(receiver, amount)) {
        cout << "Transfer successful! " << "\n";

        cout << "Do you want to print the receipt? (y/n): ";
        char confirm;
        cin >> confirm;
        if (confirm == 'y' || confirm == 'Y') {
            cout << endl;
            cout << "Date: " << __DATE__ << "\n";
            cout << "Time: " << __TIME__ << "\n";
            cout << "Sender Account Number: " <<
user.accountNumber << "\n";
            cout << "Recipient Account Number: " <<
receiver.accountNumber << "\n";
            cout << "Amount Transferred: $" << fixed <<
setprecision(2) << amount << "\n";

```

```
165             cout << "New Balance: $" << fixed << setprecision(2)
166<< user.balance << "\n";
167         }
168     } else {
169         cout << "Insufficient balance for transfer.\n";
170     }
171
172     waitforUser();
173}
174
175void payment(User &user) {
176     system("cls");
177     cout << "----------------------Payment-----------------------
178---\n";
179
180     double amount;
181     string pin;
182
183     cout << "Payment Options:\n";
184     cout << "1. Electricity bill\n";
185     cout << "2. Water bill\n";
186     cout << "3. Telephone bill\n";
187     cout << "4. Credit card bill\n";
188     cout << "Enter your option: ";
189     int option;
190     cin >> option;
191
192     string utilityCompany;
193     string billType;
194     int billNumber;
195     double totalBill = rand() % 200;
196
197     switch (option) {
198     case 1:
199         utilityCompany = "Tenaga Nasional Berhad";
200         billType = "Electricity bill";
201         billNumber = rand() % 1000000;
202         break;
203     case 2:
204         utilityCompany = "Ranhill Saj";
205         billType = "Water bill";
206         billNumber = rand() % 1000000;
207         break;
208     case 3:
209         utilityCompany = "CelcomDigi";
210         billType = "Telephone bill";
211         billNumber = rand() % 1000000;
212         break;
213     case 4:
214         utilityCompany = "Equinox Bank Berhad";
215         billType = "Credit card bill";
216         billNumber = rand() % 1000000;
217         break;
218     default:
219         cout << "Invalid option.\n";
220         return;
```

```
221     }
222
223     cout << "\nBill number: " << billNumber << "\n";
224     cout << "Total bill: RM" << totalBill << "\n";
225     cout << "Utility company: " << utilityCompany << "\n";
226     cout << "Confirmation (y/n): ";
227     char confirm;
228     cin >> confirm;
229
230     if (confirm == 'y' || confirm == 'Y') {
231         cout << "\nPayment successful!\n";
232         cout << "Date: " << __DATE__ << "\n";
233         cout << "Time: " << __TIME__ << "\n";
234         cout << "Account Number: " << user.accountNumber << "\n";
235         cout << "Bill Number: " << billNumber << "\n";
236         cout << "Amount paid: RM" << totalBill << "\n";
237         cout << "Utility company: " << utilityCompany << "\n";
238         cout << "Thank you for payment!\n";
239         user.payBill(totalBill);
240         cout << "New balance: $" << user.balance << "\n";
241     } else {
242         cout << "Payment cancelled.\n";
243     }
244
245     waitforUser();
246 }
247
248 void withdraw(User &user) {
249     double amount;
250     string pin;
251     system("cls");
252     cout << "----------------------Withdraw----------------------
253 ----\n";
254
255     srand(time(0)); // Seed the random number generator with the
256 current time
257     int atmSerialNumber = rand() % 10000000000; // Generate a
258 random 10-digit number
259
260     cout << "Enter amount to withdraw (you may withdraw up to $"
261 << fixed << setprecision(2) << min(user.balance - 10,
262 user.withdrawalLimit) << "):   $";
263     cin >> amount;
264     if (amount > user.withdrawalLimit) {
265         cout << "Amount exceeds the withdrawal limit of $" <<
266 user.withdrawalLimit << ".\n";
267         return;
268     }
269     cout << "Please scan the ATM QR code with your device.\n";
270     cout << "ATM Serial Number: " << atmSerialNumber << endl; //
271 Display the random ATM serial number
272     cout << "Confirm connection (y/n): ";
273     char confirm;
274     cin >> confirm;
275
276     if (confirm != 'y' && confirm != 'Y') {
```

```
277         cout << "Connection failed.\n";
278         return;
279     }
280 
281     cout << "Successful connection.\n";
282     cout << "Enter your security PIN: ";
283     cin >> pin;
284 
285     if (!user.authenticate(pin)) {
286         cout << "Security PIN authentication failed.\n";
287         return;
288     }
289 
290     if (user.withdraw(amount)) {
291         cout << "Withdrawal successful! New balance: $" <<
292user.balance << "\n";
293         cout << "Do you want to print the receipt? (y/n): ";
294         cin >> confirm;
295         if (confirm == 'y' || confirm == 'Y') {
296             cout << "\nWithdrawal Date: " << __DATE__ << "\n";
297             cout << "Withdrawal Time: " << __TIME__ << "\n";
298             cout << "Account Number: " << user.accountNumber <<
299"\n";
300             cout << "Amount Withdrawn: $" << amount << "\n";
301             cout << "New Balance: $" << user.balance << "\n";
302 
303             int intAmount = static_cast<int>(amount);
304             int num100 = intAmount / 100;
305             intAmount %= 100;
306             int num50 = intAmount / 50;
307             intAmount %= 50;
308             int num20 = intAmount / 20;
309             intAmount %= 20;
310             int num10 = intAmount / 10;
311             intAmount %= 10;
312             int num5 = intAmount / 5;
313             intAmount %= 5;
314             int num1 = intAmount;
315 
316             cout << "Denominations: \n";
317             cout << "RM100: " << num100 << "\n";
318             cout << "RM50: " << num50 << "\n";
319             cout << "RM20: " << num20 << "\n";
320             cout << "RM10: " << num10 << "\n";
321             cout << "RM5: " << num5 << "\n";
322             cout << "RM1: " << num1 << "\n";
323         }
324     } else {
325         cout << "Insufficient balance for withdrawal.\n";
326     }
327 
328     waitforUser();
329}
330 
331void accountSetting(User &user) {
332     system("cls");
```

```
333     cout << "----------------------Account Settings--------------
334------------\n";
335 
336     int choice;
337     cout << "Choose an option:\n";
338     cout << "1. Change Credit Card PIN\n";
339     cout << "2. Change Withdrawal Limit\n";
340     cout << "Enter your choice: ";
341     cin >> choice;
342 
343     if (choice == 1) {
344         string newPin;
345         cout << "Enter new PIN: ";
346         cin >> newPin;
347         user.setCreditCardSettings(newPin);
348     } else if (choice == 2) {
349         double newLimit;
350         cout << "Enter new withdrawal limit: $";
351         cin >> newLimit;
352         user.setWithdrawalLimit(newLimit);
353     } else {
354         cout << "Invalid option.\n";
355     }
356 
357     waitforUser();
358}
359 
360void loginUser() {
361     string userID, password, pin;
362 
363     cout << "Enter User ID: ";
364     cin >> userID;
365     cout << "Enter Password: ";
366     cin >> password;
367     cout << "Enter Security PIN: ";
368     cin >> pin;
369 
370     User user = User::loadUser(userID, password);
371     if (user.userID.empty() || !user.authenticate(pin)) {
372         cout << "Login failed: Invalid username, password, or
373security PIN.\n";
374         cout << "Wait for 2 seconds to return to the main
375menu...";
376         this_thread::sleep_for(chrono::seconds(2));
377         system("cls");
378         return;
379     }
380 
381     cout << "Login successful!\n\n\n";
382     for (int i = 3; i > 0; --i) {
383         cout << "Redirecting to main menu... " << i << " s" <<
384endl;
385         this_thread::sleep_for(chrono::seconds(1));
386     }
387 
388     system("cls");
```

```
389
390     int choice;
391     do {
392         showMenu();
393         cin >> choice;
394         switch (choice) {
395         case 1:
396             accountDetails(user);
397             break;
398         case 2:
399             deposit(user);
400             break;
401         case 3:
402             transfer(user);
403             break;
404         case 4:
405             payment(user);
406             break;
407         case 5:
408             withdraw(user);
409             break;
410         case 6:
411             accountSetting(user);
412             break;
413         case 7:
414             cout << "Exiting...\n\n\n";
415             for (int i = 3; i > 0; --i) {
416                 cout << "Redirecting to login page... " << i << "
417s" << endl;
418                 this_thread::sleep_for(chrono::seconds(1));
419             }
420             system("cls");
421             break;
422         default:
423             cout << "Invalid choice. Please try again.\n";
424         }
425     } while (choice != 7);
426}
427
428void viewUserAccounts() {
429     ifstream file("users.dat");
430     if (!file) {
431         cout << "\n\nError loading user database...\n";
432         return;
433     }
434
435     const int MAX_USERS = 20;
436     User users[MAX_USERS];
437     int numUsers = 0;
438     system("cls");
439     cout << "----------------------User Accounts-----------------
440---------\n";
441
442     while (numUsers < MAX_USERS && file >>
443users[numUsers].userID >> users[numUsers].password >>
444
```

```
445users[numUsers].accountNumber >> users[numUsers].accountType >>
446users[numUsers].balance >> users[numUsers].securityPIN) {
447         numUsers++;
448     }
449
450     file.close();
451
452     cout << "User Accounts:\n";
453     cout << "-----------------------------------------\n";
454     cout << "Account Number      | Account Type | Balance\n";
455     cout << "-----------------------------------------\n";
456     for (int i = 0; i < numUsers; i++) {
457         cout << setw(18) << users[i].accountNumber << " | " <<
458setw(12) << users[i].accountType << " | $" << setw(7) << fixed <<
459setprecision(2) << users[i].balance << "\n";
460     }
461     cout << "-----------------------------------------\n";
462
463     waitforUser();
464}
465
466void loginAdmin() {
467     string adminID, password, pin;
468
469     cout << "Enter Admin ID: ";
470     cin >> adminID;
471     cout << "Enter Password: ";
472     cin >> password;
473     cout << "Enter Security PIN: ";
474     cin >> pin;
475
476     try {
477         Admin admin = Admin::loadAdmin(adminID, password);
478         if (!admin.authenticate(pin)) {
479             throw runtime_error("Invalid security PIN.");
480         }
481
482         cout << "Admin login successful!\n\n\n";
483         for (int i = 3; i > 0; --i) {
484             cout << "Redirecting to main menu... " << i << " s"
485<< endl;
486             this_thread::sleep_for(chrono::seconds(1));
487         }
488
489         system("cls");
490
491         int choice;
492         do {
493             cout << "Main Menu:\n";
494             cout << "1. View list of users\n";
495             cout << "2. View user details\n";
496             cout << "3. Check transaction history\n";
497             cout << "4. Exit\n";
498             cout << "\nEnter your choice: ";
499             cin >> choice;
500             switch (choice) {
```

```
            case 1:
                viewUserAccounts();
                break;
            case 2: {
                string userID;
                cout << "Enter User ID to view details: ";
                cin >> userID;
                admin.viewUserDetails(userID);
                waitforUser();
                break;
            }
            case 3:
                admin.checkTransactionHistory();
                waitforUser();
                break;
            case 4:
                cout << "Exiting...\n\n\n";
                for (int i = 3; i > 0; --i) {
                    cout << "Redirecting to login page... " << i
<< " s" << endl;
                    this_thread::sleep_for(chrono::seconds(1));
                }
                system("cls");
                break;
            default:
                cout << "Invalid choice. Please try again.\n";
            }
        } while (choice != 4);
    } catch (const runtime_error &e) {
        cout << "Login failed: " << e.what() << "\n";
        this_thread::sleep_for(chrono::seconds(2));
        system("cls");
    }
}

int main() {
    int option;
    do {
        cout << "Welcome to Equinox Bank System!\n\n\n";
        cout << "1. Login (User)\n";
        cout << "2. Login (Admin)\n";
        cout << "3. Exit\n\n\n";
        cout << "Enter your choice: ";
        cin >> option;
        switch (option) {
        case 1:
            loginUser();
            break;
        case 2:
            loginAdmin();
            break;
        case 3:
            cout << "Exiting...\n";
            break;
        default:
            cout << "Invalid choice. Please try again.\n";
```

```
557             }
558         } while (option != 3);
559
            return 0;
        }
```

Source code for User class

```
#ifndef USER_H
#define USER_H

#include <iostream>
#include <fstream>
#include <sstream>
#include <string>
#include <ctime>
#include <iomanip>
using namespace std;

class User {
private:
    string userID;
    string password;
    string accountNumber;
    string accountType;
    double balance;
    string securityPIN;
    double withdrawalLimit = 2000.0; // Default withdrawal limit

public:
    User() {}
    User(string userID, string password, string accountNumber,
string accountType, double balance, string securityPIN)
        : userID(userID), password(password),
accountNumber(accountNumber), accountType(accountType),
balance(balance), securityPIN(securityPIN) {}

    void deposit(double amount) {
        balance += amount;
        saveUserData();
    }

    bool withdraw(double amount) {
        if (balance >= amount && amount <= withdrawalLimit) {
            balance -= amount;
            saveUserData();
            return true;
        }
        return false;
    }

    bool transfer(User &receiver, double amount) {
        if (balance >= amount) {
            balance -= amount;
            receiver.deposit(amount);
            saveUserData();
            receiver.saveUserData();

            // Log the transaction
            ofstream outfile("transactions.dat", ios::app);
            time_t now = time(0);
            tm *ltm = localtime(&now);
```

```
55              outfile << accountNumber << " "
56                      << 1900 + ltm->tm_year << "-" << 1 +
57 ltm->tm_mon << "-" << ltm->tm_mday << " "
58                      << 1 + ltm->tm_hour << ":" << 1 + ltm->tm_min
59 << ":" << 1 + ltm->tm_sec << " "
60                      << amount << " "
61                      << receiver.accountNumber << endl;
62              outfile.close();
63
64              return true;
65          }
66          return false;
67      }
68
69      void payBill(double amount) {
70          if (balance >= amount) {
71              balance -= amount;
72              saveUserData();
73          } else {
74              cout << "Insufficient balance." << endl;
75          }
76      }
77
78      void setCreditCardSettings(string newPin) {
79          if (!newPin.empty()) {
80              securityPIN = newPin;
81          }
82          cout << "Credit card PIN updated: New PIN: " <<
83 securityPIN << endl;
84      }
85
86      void setWithdrawalLimit(double newLimit) {
87          withdrawalLimit = newLimit;
88          cout << "New withdrawal limit: $" << withdrawalLimit <<
89 endl;
90      }
91
92      static User loadUser(const string &userID, const string
93 &password) {
94          ifstream infile("users.dat");
95          string line, id, pass, accountNumber, accountType, pin;
96          double bal;
97          while (getline(infile, line)) {
98              istringstream iss(line);
99              iss >> id >> pass >> accountNumber >> accountType >>
100 bal >> pin;
101             if (id == userID && pass == password) {
102                 infile.close();
103                 return User(id, pass, accountNumber, accountType,
104 bal, pin);
105             }
106         }
107         infile.close();
108         return User("", "", "", "", 0.0, "");
109     }
110
```

```
111     static User loadUser(const string &accNum) {
112         ifstream infile("users.dat");
113         string line, id, pass, accountNumber, accountType, pin;
114         double bal;
115         while (getline(infile, line)) {
116             istringstream iss(line);
117             iss >> id >> pass >> accountNumber >> accountType >>
118 bal >> pin;
119             if (accountNumber == accNum) {
120                 infile.close();
121                 return User(id, pass, accountNumber, accountType,
122 bal, pin);
123             }
124         }
125         infile.close();
126         return User("", "", "", "", 0.0, "");
127     }
128 
129     void saveUserData() {
130         ifstream infile("users.dat");
131         ofstream outfile("temp.txt");
132         string line;
133 
134         while (getline(infile, line)) {
135             istringstream iss(line);
136             string id, pass, accountNumber, accountType, pin;
137             double bal;
138             iss >> id >> pass >> accountNumber >> accountType >>
139 bal >> pin;
140 
141             if (id == userID) {
142                 outfile << userID << " " << password << " " <<
143 accountNumber << " " << accountType << " " << balance << " " <<
144 securityPIN << endl;
145             } else {
146                 outfile << line << endl;
147             }
148         }
149         infile.close();
150         outfile.close();
151 
152         // Remove the original file and rename the temporary file
153         remove("users.dat");
154         rename("temp.txt", "users.dat");
155     }
156 
157     bool authenticate(string pin) {
158         return pin == securityPIN;
159     }
160 
161     friend void accountDetails(User &);
162     friend void transfer(User &);
163     friend void withdraw(User &);
164     friend void loginUser();
165     friend void payment(User &);
166     friend void viewUserAccounts();
```

```
167 };
168
169 #endif
```

Source code for Admin class

```
#ifndef ADMIN_H
#define ADMIN_H

#include <iostream>
#include <fstream>
#include <sstream>
#include <string>
#include <iomanip>
using namespace std;

class Admin {
private:
    string adminID;
    string adminPassword;
    string securityPIN;

public:
    Admin(string adminID, string adminPassword, string
securityPIN)
        : adminID(adminID), adminPassword(adminPassword),
securityPIN(securityPIN) {}

    static Admin loadAdmin(const string &adminID, const string
&adminPassword) {
        ifstream infile("admins.dat");
        string line, id, pass, pin;
        while (getline(infile, line)) {
            istringstream iss(line);
            iss >> id >> pass >> pin;
            if (id == adminID && pass == adminPassword) {
                infile.close();
                return Admin(id, pass, pin);
            }
        }
        infile.close();
        throw runtime_error("Invalid admin credentials.");
    }

    bool authenticate(string pin) {
        return pin == securityPIN;
    }

    void viewUserDetails(const string &userID) {
        ifstream infile("users.dat");
        string line, id, pass, accountNumber, accountType, pin;
        double bal;
        bool userFound = false;

        while (getline(infile, line)) {
            istringstream iss(line);
            iss >> id >> pass >> accountNumber >> accountType >>
bal >> pin;
            if (id == userID) {
                cout << "User ID: " << id << "\n";
```

```
                cout << "Account Number: " << accountNumber <<
"\n";
                cout << "Account Type: " << accountType << "\n";
                cout << "Balance: $" << fixed << setprecision(2)
<< bal << "\n";
                cout << "Security PIN: " << pin << "\n";
                userFound = true;
                break;
            }
        }
        infile.close();

        if (!userFound) {
            cout << "User not found.\n";
        }
    }

    void checkTransactionHistory() {
        ifstream infile("transactions.dat");
        string line;

        cout << "Transaction History:\n";
        cout << "-----------------------------------------------
----------------------\n";
        cout << "Sender Account | Date       | Time     | Amount
| Recipient Account\n";
        cout << "-----------------------------------------------
----------------------\n";
        while (getline(infile, line)) {
            istringstream iss(line);
            string senderAcc, date, time, recipientAcc;
            double amount;
            iss >> senderAcc >> date >> time >> amount >>
recipientAcc;
            cout << setw(15) << senderAcc << " | "
                 << setw(10) << date << " | "
                 << setw(8) << time << " | "
                 << setw(7) << fixed << setprecision(2) << amount
<< " | "
                 << setw(17) << recipientAcc << "\n";
        }
        cout << "-----------------------------------------------
----------------------\n";
        infile.close();
    }
};

#endif
```

---- END OF DOCUMENTATION -----